

An Agent-Based Study Assistant System

with OCR, Retrieval-Augmented Generation, and Text-to-Speech Tools

CANDIDATE

Antonis Geralis

PROJECT ADVISOR

Ioannis Katakis

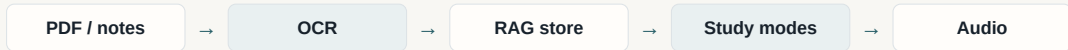
DEGREE PROGRAM

Bachelor of Science in Computer Science

Department of Computer Science
School of Sciences and Engineering
University of Nicosia

Problem Context

Study material is fragmented across formats and revision tasks



- Converts difficult source material into usable text
- Stores prepared content for semantic search
- Generates study artefacts: notes, flashcards, quizzes, essays
- Produces listenable .opus revision audio
- Keeps OCR, retrieval, generation, and speech as distinct responsibilities

Core thesis

AI study support is most useful when it is source-grounded, modular, and inspectable rather than a single unstructured prompt.

Problem Statement

Input readiness

Scanned slides, textbook pages, and PDFs are often visually readable but not directly searchable or usable by language tools.

Grounded access

Long or fragmented notes exceed convenient prompt use and need retrieval from the student's own sources.

Output form

Effective revision needs several forms: explanations, active recall, essay practice, and audio.

Design challenge

Build a study assistant that moves from raw course material to reliable, reusable study artefacts without losing source grounding.

Objectives

Functional objectives

- Support transcription, notes, lecture explanation, ELI5, flashcards, mind maps, quizzes, and essays
- Use OCR when source material is PDF-based or scanned
- Store and retrieve semantic chunks from prepared material
- Convert selected material into natural spoken audio

Engineering objectives

- Preserve deterministic ordering despite asynchronous model calls
- Bound concurrency and retry transient failures
- Keep stdout artefacts separate from diagnostics
- Make OCR, RAG, and TTS usable both inside and outside the agent

Research Questions and Scope

Research questions

- How can an agent coordinate OCR, retrieval, generation, and speech without hiding the processing steps?
- Which software contracts are needed to make model-based study workflows auditable?
- What operational evidence shows that the system is practical for realistic revision material?

Scope

The project evaluates system design, reliability, throughput, and recorded study workflows.

Boundary

It does not claim measured grade improvement or replace student judgement over generated artefacts.

Foundation

Background, requirements, and the study-workflow framing

Background and Related Work

OCR

Converts visual document pages into machine-readable text.

Evaluation must consider not only CER/WER but also reading order, tables, equations, and recall.

RAG

Combines language generation with retrieved passages from an external corpus, allowing answers and artefacts to remain tied to source material.

TTS

Turns prepared text into audio, but requires speech-oriented rewriting so markdown, formulae, and technical notation remain understandable.

Positioning

Existing AI study tools often focus on summarisation. This project treats study assistance as a coordinated lifecycle: prepare, retrieve, transform, practise, and listen.

Requirements and Specification

User-facing requirements

- Work from PDFs, text, Markdown, stored document IDs, or raw source text
- Produce concise, mode-specific study artefacts
- Avoid unsupported external content during transformations
- Make intermediate artefacts auditable

Tool contracts

Tool	Contract
pdfocr	Ordered JSONL page results
chunkvec	Marked chunks -> SQLite vector store
chunktts	Ordered chunks -> complete .opus file

Method and System Design

Evaluation method, architecture, and auditable artefact contracts

Methodology

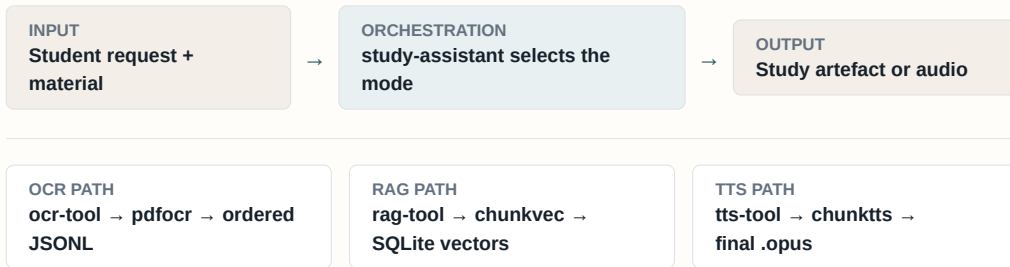
Common model-calling pattern

1. Parse and normalise CLI/configuration
2. Convert input into ordered work items
3. Assign deterministic request IDs
4. Submit bounded batches through relay
5. Classify success, retryable failure, or terminal failure
6. Finalise in source order or database transaction

Correctness invariants

- `inFlightCount` $\leq K$
- request ID encodes sequence and attempt
- retry queues are ordered by due time
- stdout remains machine-readable
- API schemas are built through typed helpers
- configuration values are normalised into safe ranges

System Architecture



Shared libraries provide bounded HTTP execution, typed JSON handling, and provider request schemas.

The visual split is deliberate: the agent owns study intent; the tools own deterministic processing contracts.

Design Decisions

1. Agent orchestration, deterministic tools

The agent handles study intent. Core tools handle stable execution contracts, retries, schemas, and artefact publication.

2. Standalone command-line tools

Each processing stage can be inspected, tested, redirected, cached, or reused without the full assistant.

3. Ordered output over raw speed

Results may complete out of order, but OCR pages and audio chunks are published in source order.

4. Explicit failure semantics

Partial OCR is auditable; partial final audio is withheld because it would look complete while omitting content.

Implementation Highlights

pdfocr

PDFium rendering, WebP encoding, multimodal OCR requests, page-level JSONL, retry/error classification.

chunkvec

Strict `<chunk . . .>` parser, embeddings pipeline, SQLite storage, metadata filters, vector search.

chunktts

`<bk>` chunk splitting, speech API requests, WAV decoding, audio validation, final Opus assembly.

relay

Worker thread over libcurl multi; bounded concurrent request execution.

jsonx

Centralised typed JSON parsing and streaming writers.

openai

Transport-transparent request schemas for chat, embeddings, and speech.

User Workflow

A REALISTIC REVISION PATH



Supported modes

- transcribe: preserve source text
- lecture: formal explanation
- eli5: simpler explanation
- flashcard: active recall
- mindmap: concept hierarchy
- quiz: practice questions
- essay: exam prompts
- study-notes: revision notes

Demo scenario

- Use OCR on lecture slides
- Generate study notes
- Rewrite notation and headings for speech
- Synthesize 24 ordered chunks
- Publish one .opus revision artefact

Recorded Workflow Protocol

DEMONSTRATION PROTOCOL

1. Select a lecture PDF
2. Run OCR or use cached extraction
3. Ask for a specific mode, e.g. flashcards
4. Show output grounded in extracted content
5. Store a textbook section in RAG
6. Search for a targeted concept
7. Convert generated notes into audio

Interpretation focus

The evidence is not one generated answer. Each step creates an inspectable artefact, and later stages reuse the student's own material.

Recorded evidence

Workflows were recorded for essay practice, RAG exam revision, flashcards, and study-notes-to-audio.

Evaluation

Contract tests, operational benchmarks, and recorded study workflows

Testing Strategy

Verified software contracts

- `pdfocr`: page selection, request IDs, retry queue, JSON schema
- `chunkvec`: chunk parser, config, embeddings, SQLite/vector integration
- `chunktts`: splitting, retries, audio wrapper, local-server pipeline test
- `relay`: lifecycle, ordering, request bodies, headers
- `jsonx` and `openai`: parser and schema tests

Why this matters

- Separates implementation correctness from provider/model variability
- Tests deterministic invariants locally
- Treats live model benchmarks as empirical operational evidence
- Makes failure modes visible through stable exit codes and artefacts

Evaluation: OCR Throughput

72-PAGE SLIDE PDF BENCHMARK

**Bounded concurrency turns
OCR from a waiting problem
into an ordered batch process.**

15.89x

speedup

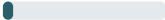
bounded concurrency vs sequential

93.71%

relative time reduction

296.73 seconds saved

RUNTIME COMPARISON

Sequential K=1		316.66
Concurrent K=32		19.93

Config	Runtime	Result
K=1	316.66 s	72/72 ok
K=32	19.93 s	72/72 ok

All pages succeeded in both runs; the improvement comes from overlapping network-bound OCR requests.

Evaluation: OCR Model Benchmark

ACCURACY-ORIENTED COMPARISON

Model	CER	WER	Order F1	Math F1
PaddleOCR-VL	0.4634	0.4732	0.3751	0.7248
olmOCR 2	0.4682	0.4893	0.3252	0.6743
DeepSeek-OCR	0.5862	0.6512	0.1452	0.4684

COST COMPARISON

Model	Total cost	Cost/page
PaddleOCR-VL	\$0.0420	\$0.0006180
olmOCR 2	\$0.0214	\$0.0003142
DeepSeek-OCR	\$0.0041	\$0.0000597

Dataset

68 pages from 34 academic PDFs with locked human gold labels; includes equations, tables, diagrams, and multi-column layouts.

Interpretation

PaddleOCR-VL wins strict accuracy. olmOCR 2 is selected for recall-oriented study workflows. DeepSeek-OCR is strongest on cost.

Workflow Evaluation Results

Essay practice

Association Analysis slides produced 4 exam-style essay questions with sample answers covering Apriori, support, confidence, lift, and interestingness.

Flashcards

Anomaly Detection slides produced 25 front/back flashcards covering definitions, settings, measures, limitations, and applications.

RAG exam revision

Textbook pages 358-402 were OCR-processed and stored as 26 semantic chunks; search retrieved zero-probability handling for Naive Bayes.

Study notes to audio

Clustering notes became a 7.3 KB Markdown artefact, a 7.2 KB TTS input, and a 2.1 MB .opus file from 24 speech chunks.

Discussion and Limitations

Interpretation

- Modularity makes AI study workflows easier to inspect and test
- Artefact boundaries are a practical reliability mechanism
- Bounded concurrency is valuable for network-bound OCR
- Retrieval quality depends on disciplined chunking and metadata

Limitations

- No controlled study of learning outcomes or retention
- OCR and generation quality still depend on external models
- RAG evaluation focuses on retrieval infrastructure, not answer grading
- Recorded workflows demonstrate feasibility rather than broad deployment

Contributions

Conceptual

Frames study assistance as a source-grounded workflow from raw material to revision artefacts.

Architectural

Separates agent orchestration, tool definitions, core Nim executables, and shared libraries.

Technical

Implements OCR, semantic retrieval, and speech pipelines with deterministic ordering, retries, schemas, and exit contracts.

Evaluation

Provides contract tests, throughput evidence, scientific OCR model comparison, and recorded student workflow artefacts.

Future Work

Engineering extensions

- Deterministic mocked transport tests across all model-calling tools
- Broader OCR benchmark corpora and repeated measurements
- Richer retrieval evaluation with labelled queries
- Provider/model selection policies per cost, latency, and privacy

Educational extensions

- Human evaluation of study artefact usefulness
- Learning-outcome studies for retention and exam preparation
- Local/private model backends for sensitive course material
- Additional modes for accessibility and instructor-authored rubrics

Conclusion

Main conclusion

study-assistant demonstrates that **OCR, RAG, and TTS** can be integrated into a practical, inspectable study workflow when the system is built around source grounding and explicit processing contracts.

- The architecture is modular and reusable
- The implementation is testable through artefact contracts
- Bounded concurrency gives a measured OCR throughput gain
- Recorded workflows show useful student-facing transformations

72/72

OCR pages succeeded
throughput benchmark

26

semantic RAG chunks
textbook revision workflow

24

speech chunks
notes-to-audio workflow

Questions

An Agent-Based Study Assistant System with OCR, Retrieval-Augmented Generation, and Text-to-Speech Tools

Antonis Geralis

Department of Computer Science, University of Nicosia